Direction Régionale Drâa Tafilalet

Examen de Fin de Module Régional N°204

« Développement Frontend »

Année : 2025/2026

Filière : Développement Digital (Option : Web Full Stack) **Epreuve** : Pratique (variante N°2)

Niveau : TS **Barème** : /40

Durée : 2H30

Partie théorique	Note
Dossier 1 : *Question à Choix Unique :*	(5pts)
1. Quelle méthode est obligatoire dans un composant de classe ?	(1pt)
A. render()	
B. show()	
C. display()	
D. view()	
2. Comment déclarer une variable d'état avec useState ?	(1pt)
A. const [count, setCount] = useState(0)	
B. const count = useState(0)	
C. useState(count, 0)	
D. const count = useHook(0)	
3. Quelle méthode est appelée après chaque mise à jour du composant ?	(1pt)
A. componentDidMount()	
B. componentDidUpdate()	
C. componentWillUpdate()	
D. shouldComponentUpdate()	

4. ***Pour exécuter un effet une seule fois :*** A. useEffect(callback) B. useEffect(callback, []) C. useEffect([], callback) D. useOnce(callback)	(1pt)
5. ***Un reducer prend :*** A. (state, action) B. (action, payload) C. (store, action) D. (state, store)	(1pt)
Partie pratique :	***Note***
Dossier 1 : ***Mini Application (FoodApp)*** **Description :** L'application **FoodApp** permet d'afficher une liste de plats avec leurs informations principales : • **nom du plat**, **cuisine**, **prix**, **note**. Chaque plat est présenté sous forme de **carte** avec un bouton « **Like** » qui incrémente un compteur local. L'application contient : • une **barre de navigation (Header)** avec deux liens : *Accueil* et *Détails*, • une **section principale (DishList)** affichant les plats, • un **pied de page (Footer)**, • une **page Détails** simulant la navigation. Les données des plats sont récupérées via Axios depuis une **API Mocki** : ▪ **https://mocki.io/v1/fcbb686a** Fetched_Data ==> [{"id": 1, "nom": "Couscous Royal", "cuisine": "Marocaine","prix": 90,"note": 4.8 }, {"id": 2, "nom": "Tajine de poulet", "cuisine": "Marocaine", "prix": 70, "note": 4.6}, {"id": 3, "nom": "Pizza Margherita", "cuisine": "Italienne", "prix": 60, "note": 4.3}, {"id": 4, "nom": "Sushi Mix", "cuisine": "Japonaise", "prix": 120, "note": 4.7}]	(20pts)

(1pt)

(2pts)

(1pt)

(2pts)

NB : Utiliser des styles inline ou du CSS externe pour le design !!

1. Quelle est la commande permettant de créer un projet React nommé movieapp ? (2pts)
2. Quelles sont les commandes permettant de se déplacer dans le dossier du projet après sa création et de lancer l'application React dans le navigateur ? (4pts)
3. Donner la commande pour installer les deux bibliothèques : **react-router-dom**, et **axios**. (4pts)
4. Créer le composant **Header.js** avec :
   - le style suivant : (background-color: #006400; color: white; padding: 15px;)
   - un menu de navigation permettant de se déplacer entre les pages Accueil et Détails (avec <Link> de react-router-dom).
5. Créer le composant **Footer.js**. (4pts)
   - le style : (background-color: #006400; color: white; padding: 10px; text-align: center;)
6. Créer le composant DishCard.js
   - Chaque DishCard gère un likeCount avec un bouton pour l'incrémenter (useState).
   - Style : (border, border-radius, margin, box-shadow, width, padding...)
7. Créer DishList.js qui utilise axios.get() pour récupérer la liste des plats
   - API : https://mocki.io/v1/fcbb686a-4766-4a08-90e3-934ef65dc33b
   - – Utiliser .map() pour générer automatiquement plusieurs cartes <DishCard/>
8. Dans App.js, utiliser React Router pour naviguer entre :
   - / (liste des plats → http://localhost:3000/)
   - **/details** (détails d'un plat → http://localhost:3000/details)
   - Créer le composant DishDetail.js pour afficher une page "Détails" simple pour simuler la navigation.
   - Importer et rendre le composant dans App.js.

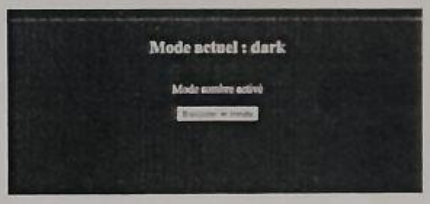
(15 pts)

Dossier 2 : Redux – Gestion du Thème (ThemeApp)

L'application ThemeApp contient un bouton qui permet à l'utilisateur de basculer entre deux thèmes : **(1pt)**

- Light mode (fond blanc, texte noir) , avec les propriétés css **backgroundColor** et **color** **(3pts)**
- Dark mode (fond noir, texte blanc) aussi avec les propriétés css **backgroundColor** et **color** **(3pts)**

Le thème choisi est géré dans le store Redux. **(6pts)**



(2pts)

1. Donner la commande pour installer la bibliothèque ***Redux*** ?
2. Créez un slice Redux nommé themeSlice
   - Le state initial doit être { mode: "light" }
   - Ajoutez une action toggleTheme qui change le mode entre "light" et "dark".
3. Créez et configurez le store Redux :
   - Enregistrez le reducer **theme: themeReducer**
4. Créez le composant ThemeSwitcher.js :
   - Utilisez **useSelector** pour lire **state**
   - Utilisez **useDispatch** pour appeler **toggleTheme()**
   - Ajoutez un bouton "Basculer le mode"
   - Utilisez un style inline qui change le fond et la couleur du texte selon le thème.
5. Fournissez le **store** à l'application dans index.js